

# RAPPORT TECHNIQUE COMPLET — APPLICATION PERFTTRACK

## Système de Gestion des Compétences et d'Évaluation des Performances

Rédigé intégralement en FRANÇAIS | Basé sur l'analyse du code source réel Stack : MERN (MongoDB · Express · React · Node.js) | Auteur : Analyse automatique du dépôt Moatazbench/PFE

## TABLE DES MATIÈRES

- [Présentation Générale de l'Application](#)
- [Architecture Backend](#)
- [Architecture Frontend](#)
- [Authentification & Autorisation](#)
- [Communication Frontend ↔ Backend](#)
- [Fonctionnalités Métier Principales](#)
- [Sécurité](#)
- [Déploiement & Infrastructure](#)
- [Questions de Jury — Avec Réponses Détaillées](#)
- [Ce que le Développeur doit Maîtriser](#)

## 1. PRÉSENTATION GÉNÉRALE DE L'APPLICATION

### 1.1 Nom et Rôle de l'Application

L'application s'appelle **PerfTrack** (visible dans le composant `Login.jsx` : `<h1>PerfTrack</h1>`).

**Problème résolu** : PerfTrack est un **système d'évaluation des performances et de gestion des compétences** destiné à une organisation bancaire (BIAT — Banque Internationale Arabe de Tunisie, comme l'indique la contrainte d'email `@biat.com`). L'application couvre l'intégralité du cycle de vie de la performance annuelle d'un collaborateur :

- Définition et suivi des **objectifs SMART** avec KPIs
- Cycle annuel en 3 phases** (fixation des objectifs → bilan mi-annuel → évaluation finale)
- Évaluations formelles** avec workflow d'approbation multi-niveaux
- Gestion des équipes**, des réunions, des feedbacks et des check-ins

- **Décisions RH** (primes, promotions, plans d'amélioration)
- **Assistance IA** (Gemini / OpenAI / Grok) pour générer des synthèses de review
- **Parcours carrière** et plans de développement

## 1.2 Stack Technique Complète

### Backend

| Technologie            | Version               | Rôle                          |
|------------------------|-----------------------|-------------------------------|
| Node.js                | 22 (Docker) / 18 (CI) | Runtime JavaScript serveur    |
| Express.js             | ^4.22.1               | Framework HTTP REST           |
| MongoDB                | Atlas (cloud)         | Base de données NoSQL         |
| Mongoose               | ^7.8.9                | ODM (Object Document Mapper)  |
| bcryptjs               | ^2.4.3                | Hashage des mots de passe     |
| jsonwebtoken           | ^9.0.0                | Gestion des tokens JWT        |
| helmet                 | ^8.1.0                | Sécurité HTTP headers         |
| express-rate-limit     | ^8.2.1                | Limitation de débit           |
| xss-clean              | ^0.1.4                | Protection XSS                |
| express-mongo-sanitize | ^2.2.0                | Protection injection NoSQL    |
| joi                    | ^18.0.2               | Validation des entrées        |
| multer                 | ^2.0.2                | Upload de fichiers            |
| node-cron              | ^3.0.0                | Tâches planifiées             |
| nodemailer             | ^8.0.1                | Envoi d'emails                |
| pdfkit                 | ^0.13.0               | Génération de PDFs            |
| @google/generative-ai  | ^0.24.1               | SDK Gemini AI                 |
| openai                 | ^5.23.2               | SDK OpenAI / Grok             |
| compression            | ^1.8.1                | Compression Gzip des réponses |
| dotenv                 | ^16.6.1               | Variables d'environnement     |

### Frontend

| Technologie      | Version | Rôle                         |
|------------------|---------|------------------------------|
| React            | ^18.3.1 | Bibliothèque UI              |
| Vite             | ^7.2.4  | Outil de build et dev server |
| React Router DOM | ^6.30.3 | Routing côté client          |
| Axios            | ^1.13.3 | Client HTTP                  |
| Chart.js         | ^4.5.1  | Graphiques                   |
| Recharts         | ^3.3.0  | Graphiques React             |

| Technologie   | Version   | Rôle                   |
|---------------|-----------|------------------------|
| Framer Motion | ^12.23.24 | Animations             |
| @dnd-kit      | ^6.3.1    | Drag & Drop            |
| date-fns      | ^4.1.0    | Manipulation des dates |
| jsPDF         | ^4.2.1    | Export PDF côté client |
| html2canvas   | ^1.4.1    | Capture d'écran HTML   |

### 1.3 Structure Globale des Dossiers

|                                       |                                                  |
|---------------------------------------|--------------------------------------------------|
| application_gestion_competences/      | ← Racine du projet                               |
| ├─ backend/                           | ← API Node.js/Express                            |
| │   ├─ app.js                         | ← Configuration Express (middlewares, routes)    |
| │   └─ server.js                      | ← Point d'entrée : connexion MongoDB + démarrage |
| │   ├─ models/                        | ← 20 schémas Mongoose                            |
| │   ├─ routes/                        | ← 28 fichiers de routes                          |
| │   ├─ controllers/                   | ← 15 controllers de logique métier               |
| │   └─ middleware/                    | ← 8 middlewares (auth, role, error, etc.)        |
| │   └─ services/                      | ← 3 services (AI, score, review context)         |
| │   ├─ validators/                    | ← Schémas de validation Joi                      |
| │   ├─ utils/                         | ← Utilitaires                                    |
| │   ├─ cron/                          | ← Tâches planifiées                              |
| │   ├─ config/                        | ← Configuration                                  |
| │   ├─ uploads/                       | ← Fichiers uploadés (checkins/)                  |
| │   ├─ Dockerfile                     | ← Image Docker backend                           |
| │   └─ .env                           | ← Variables d'environnement                      |
| ├─ frontend/                          | ← Application React                              |
| │   ├─ src/                           | ← Point d'entrée React                           |
| │   │   ├─ main.jsx                   | ← Routage principal + Providers                  |
| │   │   ├─ App.jsx                    |                                                  |
| │   │   ├─ api/                       |                                                  |
| │   │   │   └─ apiClient.js           | ← Instance Axios centralisée                     |
| │   │   ├─ components/                |                                                  |
| │   │   │   ├─ AuthContext.jsx        | ← Gestion état auth global                       |
| │   │   │   ├─ ProtectedRoute.jsx     | ← Garde de route basique                         |
| │   │   │   ├─ RouteGuard.jsx         | ← Garde avancée (rôle + phase cycle)             |
| │   │   │   ├─ ThemeContext.jsx       | ← Mode sombre/clair                              |
| │   │   │   ├─ ActiveCycleContext.jsx | ← Cycle actif global                             |
| │   │   │   ├─ EnterpriseSidebar.jsx  | ← Navigation latérale                            |
| │   │   │   └─ TopHeader.jsx          | ← En-tête                                        |
| │   │   ├─ pages/                     | ← 30 pages                                       |
| │   │   ├─ routes/                    |                                                  |
| │   │   │   └─ routeConfig.jsx        | ← Configuration centralisée des routes           |
| │   │   ├─ hooks/                     | ← Hooks personnalisés                            |
| │   │   └─ services/                  | ← Services API                                   |
| │   ├─ vite.config.js                 | ← Configuration Vite + proxy                     |
| │   ├─ nginx.conf                     | ← Configuration Nginx (Docker)                   |
| │   └─ Dockerfile                     | ← Image Docker frontend (multi-stage)            |
| ├─ k8s/                               | ← Manifests Kubernetes                           |
| │   ├─ base/                          | ← Base Kustomize                                 |
| │   │   ├─ backend-deployment.yaml    |                                                  |
| │   │   └─ frontend-deployment.yaml   |                                                  |
| │   └─ overlays/                      | ← Environnements (dev, qa, staging, ...)         |

|                             |                                 |
|-----------------------------|---------------------------------|
| prod)                       |                                 |
| ├─ argocd-app.yaml          | ← Application ArgoCD            |
| ├─ .github/                 |                                 |
| │   └─ workflows/           |                                 |
| │       └─ docker-build.yml | ← Pipeline CI/CD GitHub Actions |
| └─ docker-compose.yml       | ← Orchestration locale          |

### 1.4 Points d'Entrée Principaux

| Fichier               | Rôle                                                                              |
|-----------------------|-----------------------------------------------------------------------------------|
| backend/server.js     | Charge .env, vérifie les variables requises, connecte MongoDB, démarre Express    |
| backend/app.js        | Configure tous les middlewares de sécurité, charge les 28 routes                  |
| frontend/src/main.jsx | Monte <App /> dans le DOM via ReactDOM.createRoot                                 |
| frontend/src/App.jsx  | Déclare les Providers (Auth, Theme, ActiveCycle), définit le routage React Router |

## 2. ARCHITECTURE BACKEND

### 2.1 Framework et Version

**Express.js ^4.22.1** — framework minimaliste et non-opinioné. Le point d'entrée est `server.js` qui délègue toute la configuration à `app.js`.

### 2.2 Organisation des Fichiers

|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| backend/                          |                                                          |
| ├─ app.js                         | ← Application Express : middlewares + montage des routes |
| ├─ server.js                      | ← Bootstrap : dotenv + MongoDB + listen                  |
| ├─ routes/                        | ← 28 fichiers, 1 par domaine fonctionnel                 |
| ├─ controllers/                   | ← 15 fichiers de logique métier                          |
| ├─ models/                        | ← 20 schémas Mongoose                                    |
| ├─ middleware/                    | ← 8 middlewares                                          |
| │   └─ auth.js                    | ← Vérification JWT                                       |
| │   └─ role.js                    | ← RBAC (contrôle d'accès par rôle)                       |
| │   └─ errorHandler.js            | ← Gestion globale des erreurs                            |
| │   └─ validate.js                | ← Validation Joi des corps de requête                    |
| │   └─ rateLimiter.js             | ← Limitation de débit personnalisée                      |
| │   └─ audit.js                   | ← Journalisation des actions                             |
| │   └─ ownership.js               | ← Vérification de propriété de ressource                 |
| │   └─ validateEnv.js             | ← Vérification des variables d'env au démarrage          |
| ├─ services/                      | ← 3 services métier                                      |
| │   └─ aiService.js               | ← Abstraction multi-provider IA                          |
| (Gemini/OpenAI/Grok)              |                                                          |
| │   └─ reviewContextService.js    | ← Collecte du contexte pour les reviews IA               |
| │   └─ scoreCalculationService.js | ← Calcul des scores d'évaluation                         |
| ├─ validators/                    | ← Schémas Joi de validation                              |
| ├─ cron/                          | ← Tâches planifiées (node-cron)                          |
| └─ utils/                         | ← Fonctions utilitaires partagées                        |

## 2.3 Base de Données

- **SGBD** : MongoDB (hébergé sur MongoDB Atlas — cloud)
- **ODM** : Mongoose ^7.8.9
- **URI de connexion** : définie dans la variable d'environnement MONGO\_URI

```
MONGO_URI=mongodb+srv://moataz:****@clusterhr.l7w8pww.mongodb.net/hr_evaluation?appName=ClusterHR
```

### Connexion (dans `server.js`) :

```
mongoose.connect(mongoUri, {  
  serverSelectionTimeoutMS: 5000,  
  connectTimeoutMS: 10000,  
})
```

Si la connexion échoue, le processus s'arrête (`process.exit(1)`).

## 2.4 Modèles de Données — 20 Schémas Mongoose

| # | Modèle       | Collection | Champs Principaux                                                                                                                                          | Relations                                            |
|---|--------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| 1 | <b>User</b>  | users      | name, email (@biat.com), password (select:false), role (ADMIN/HR/TEAM_LEADER/COLLABORATOR), team, manager, isActive, isDeleted, profileImage, refreshToken | → Team, → User (manager)                             |
| 2 | <b>Team</b>  | teams      | name, description, leader, members[], parentTeam                                                                                                           | → User (leader), → User[] (members), → Team (parent) |
| 3 | <b>Cycle</b> | cycles     | name, year (unique), status (draft/open/active/in_progress/closed), phase1Start/End, phase2Start/End,                                                      | → User (createdBy)                                   |

| # | Modèle                 | Collection       | Champs Principaux                                                                                                                                                                                                                                                              | Relations                                   |
|---|------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
|   |                        |                  | phase3Start/End, currentPhase                                                                                                                                                                                                                                                  |                                             |
| 4 | <b>Objective</b>       | objectives       | title, dueDate, description, successIndicator, owner, cycle, category (individual/team), weight, achievementPercent, status (draft → approved → evaluated), source (employee_created/manager_assigned), kpis[], progressUpdates[], comments[], changeRequests[], activityLog[] | → User (owner, assignedBy), → Cycle, → Team |
| 5 | <b>Evaluation</b>      | evaluations      | employeeId, evaluatorId, cycleId, status (draft → completed), objectiveAssessments[], finalScore, scoreHistory[], strengths, areasForImprovement, approvals[], employeeAcknowledgment                                                                                          | → User, → Cycle, → Objective                |
| 6 | <b>FinalEvaluation</b> | finalevaluations | employee_id, cycle_id,                                                                                                                                                                                                                                                         | → User, → Cycle                             |

| #  | Modèle              | Collection    | Champs Principaux                                                                                                                                                                          | Relations                    |
|----|---------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
|    |                     |               | auto_score, manager_score, final_score, rating_label (exceptional → unsatisfactory), recommendation (promotion/bonus/pip/etc.), status (draft → validated), hr_decision, employee_feedback |                              |
| 7  | <b>Feedback</b>     | feedbacks     | sender, recipient, type (praise/suggestion/360/peer/manager), message, anonymous, visibility, rating, relatedObjective, cycle_id                                                           | → User, → Objective, → Cycle |
| 8  | <b>CheckIn</b>      | checkins      | objective_id, employee_id, cycle_id, status (draft/pending_review/approved), progress_percent, notes, attachments[], history[]                                                             | → Objective, → User, → Cycle |
| 9  | <b>Notification</b> | notifications | recipient, sender, type (30+ types d'événements), title, message, link, isRead                                                                                                             | → User                       |
| 10 | <b>HRDecision</b>   | hrdecisions   | user, cycle, finalScore,                                                                                                                                                                   | → User, → Cycle              |

| #  | Modèle                 | Collection       | Champs Principaux                                                                                                                                                           | Relations                                |
|----|------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
|    |                        |                  | action<br>(reward/promotion<br>/bonus/coaching/te<br>rmination_review)<br>, notes,<br>decidedBy                                                                             |                                          |
| 11 | <b>ImprovementPlan</b> | improvementplans | evaluation_id,<br>employee_id,<br>cycle_id,<br>objective_goal,<br>deadline,<br>expected_outcome,<br>progress_status                                                         | →<br>FinalEvaluation,<br>→ User, → Cycle |
| 12 | <b>CareerPath</b>      | careerpaths      | user,<br>currentRole,<br>targetRole,<br>targetDate,<br>competencies<br>[] (avec<br>currentLevel/targetLevel/gap),<br>developmentPlan[],<br>mentorId,<br>status              | → User, →<br>Competency                  |
| 13 | <b>Task</b>            | tasks            | title,<br>description,<br>assignedTo,<br>createdBy,<br>team, cycle,<br>dueDate,<br>priority<br>(low/medium/high/urgent), status,<br>tags[],<br>checklist[],<br>timeTracking | → User, → Team,<br>→ Cycle               |
| 14 | <b>Meeting</b>         | meetings         | title, type,<br>organizer,<br>participants<br>[], startTime,<br>endTime,<br>agenda, notes,                                                                                  | → User, →<br>Objective                   |



| #  | Modèle                      | Collection            | Champs Principaux                                                                    | Relations            |
|----|-----------------------------|-----------------------|--------------------------------------------------------------------------------------|----------------------|
|    |                             |                       | actionItems[, objective]                                                             |                      |
| 15 | <b>AuditLog</b>             | auditlogs             | user, action, resourceType, resourceId, details (method, path, statusCode), tenantId | → User               |
| 16 | <b>BonusPenalty</b>         | bonuspenalties        | (score, type, valeur, employé, cycle)                                                | → User, → Cycle      |
| 17 | <b>CalendarConnection</b>   | calendarconnections   | (intégration calendrier externe)                                                     | → User               |
| 18 | <b>Competency</b>           | competencies          | name, description, category, levels[]                                                | —                    |
| 19 | <b>CareerRecommendation</b> | careerrecommendations | (recommandations IA de parcours)                                                     | → User               |
| 20 | <b>CorrectionRequest</b>    | correctionrequests    | (demandes de correction sur évaluation)                                              | → User, → Evaluation |

### Index composites importants :

- Objective:{ owner, cycle, title }(unique), { owner, status }, { team, cycle, category }
- Evaluation:{ employeeId, cycleId }, { evaluatorId, cycleId }
- FinalEvaluation:{ employee\_id, cycle\_id }(unique)
- HRDecision:{ user, cycle }(unique)
- Notification:{ recipient, createdAt: -1 }

## 2.5 API REST — Routes par Domaine Fonctionnel

Toutes les routes sont préfixées par `/api`. Les routes protégées nécessitent le middleware `auth` (JWT), les routes restreintes nécessitent en plus `role(...)`.

### *Authentication (/api/auth)*

| Méthode | Endpoint        | Rôle               | Protection |
|---------|-----------------|--------------------|------------|
| POST    | /api/auth/login | Connexion — génère | Public     |

| Méthode | Endpoint          | Rôle                                          | Protection |
|---------|-------------------|-----------------------------------------------|------------|
|         |                   | access + refresh token                        |            |
| POST    | /api/auth/logout  | Déconnexion — invalide le refresh token en DB | Public     |
| POST    | /api/auth/refresh | Renouvellement du token d'accès (rotation)    | Public     |
| GET     | /api/auth/me      | Récupération du profil utilisateur connecté   | JWT inline |

### *Utilisateurs & Équipes*

| Méthode | Endpoint          | Rôle                        | Protection                  |
|---------|-------------------|-----------------------------|-----------------------------|
| GET     | /api/users        | Liste tous les utilisateurs | auth + ADMIN/HR             |
| POST    | /api/users        | Créer un utilisateur        | auth + ADMIN/HR             |
| PUT     | /api/users/:id    | Modifier un utilisateur     | auth + ADMIN/HR             |
| DELETE  | /api/users/:id    | Désactiver un utilisateur   | auth + ADMIN                |
| GET     | /api/teams        | Liste des équipes           | auth                        |
| POST    | /api/teams        | Créer une équipe            | auth + ADMIN/HR             |
| PUT     | /api/teams/:id    | Modifier une équipe         | auth + ADMIN/HR             |
| GET     | /api/team-members | Membres d'une équipe        | auth                        |
| POST    | /api/team-members | Ajouter un membre           | auth + ADMIN/HR/TEAM_LEADER |

### *Cycles & Objectifs*

| Méthode | Endpoint              | Rôle                       | Protection      |
|---------|-----------------------|----------------------------|-----------------|
| GET     | /api/cycles           | Liste des cycles           | auth            |
| POST    | /api/cycles           | Créer un cycle             | auth + ADMIN/HR |
| PUT     | /api/cycles/:id       | Modifier un cycle          | auth + ADMIN/HR |
| PATCH   | /api/cycles/:id/phase | Avancer la phase du cycle  | auth + ADMIN/HR |
| GET     | /api/objectives       | Objectifs de l'utilisateur | auth            |
| POST    | /api/objectives       | Créer un objectif          | auth            |
| PUT     | /api/objectives/:id   | Modifier un objectif       | auth            |

| Méthode | Endpoint                           | Rôle                       | Protection            |
|---------|------------------------------------|----------------------------|-----------------------|
| DELETE  | /api/objectives/:id                | Supprimer un objectif      | auth                  |
| POST    | /api/objectives/:id/submit         | Soumettre pour approbation | auth                  |
| POST    | /api/objectives/:id/approve        | Approuver un objectif      | auth + TEAM_LEADER/HR |
| POST    | /api/objectives/:id/reject         | Rejeter un objectif        | auth + TEAM_LEADER/HR |
| POST    | /api/objectives/:id/evaluate       | Évaluer la réalisation     | auth + TEAM_LEADER/HR |
| POST    | /api/objectives/:id/checkin        | Ajouter un check-in        | auth                  |
| POST    | /api/objectives/:id/kpi            | Mettre à jour un KPI       | auth                  |
| POST    | /api/objectives/:id/change-request | Demander une modification  | auth                  |
| POST    | /api/objectives/:id/ai-suggest     | Suggestions IA d'objectifs | auth                  |

### **/// Évaluations & Reviews**

| Méthode | Endpoint                     | Rôle                    | Protection            |
|---------|------------------------------|-------------------------|-----------------------|
| GET     | /api/evaluations             | Liste des évaluations   | auth                  |
| POST    | /api/evaluations             | Créer une évaluation    | auth + TEAM_LEADER/HR |
| PUT     | /api/evaluations/:id         | Modifier une évaluation | auth                  |
| POST    | /api/evaluations/:id/submit  | Soumettre l'évaluation  | auth                  |
| POST    | /api/evaluations/:id/approve | Approuver l'évaluation  | auth + HR             |
| GET     | /api/final-evaluations       | Évaluations finales     | auth                  |

| Méthode | Endpoint                            | Rôle                        | Protection            |
|---------|-------------------------------------|-----------------------------|-----------------------|
| POST    | /api/final-evaluations              | Créer une évaluation finale | auth + TEAM_LEADER/HR |
| PUT     | /api/final-evaluations/:id          | Modifier éval. finale       | auth                  |
| POST    | /api/final-evaluations/:id/validate | Valider (RH)                | auth + HR             |
| GET     | /api/checkins                       | Check-ins                   | auth                  |
| POST    | /api/checkins                       | Créer un check-in           | auth                  |
| PUT     | /api/checkins/:id                   | Modifier check-in           | auth                  |

### ***RH & Décisions***

| Méthode | Endpoint               | Rôle                 | Protection      |
|---------|------------------------|----------------------|-----------------|
| GET     | /api/hr-decisions      | Décisions RH         | auth + HR/ADMIN |
| POST    | /api/hr-decisions      | Créer décision RH    | auth + HR/ADMIN |
| GET     | /api/improvement-plans | Plans d'amélioration | auth            |
| POST    | /api/improvement-plans | Créer un plan        | auth + HR/ADMIN |
| GET     | /api/bonus-penalty     | Bonus & Pénalités    | auth + HR/ADMIN |
| POST    | /api/bonus-penalty     | Créer bonus/pénalité | auth + HR/ADMIN |

### ***Social & Communication***

| Méthode | Endpoint                    | Rôle                    | Protection |
|---------|-----------------------------|-------------------------|------------|
| GET     | /api/feedback               | Liste des feedbacks     | auth       |
| POST    | /api/feedback               | Donner un feedback      | auth       |
| GET     | /api/notifications          | Notifications           | auth       |
| PUT     | /api/notifications/:id/read | Marquer comme lu        | auth       |
| GET     | /api/feed                   | Fil d'actualités équipe | auth       |

| Méthode | Endpoint      | Rôle                  | Protection |
|---------|---------------|-----------------------|------------|
| GET     | /api/meetings | Réunions              | auth       |
| POST    | /api/meetings | Créer une réunion     | auth       |
| GET     | /api/tasks    | Tâches                | auth       |
| POST    | /api/tasks    | Créer une tâche       | auth       |
| GET     | /api/calendar | Événements calendrier | auth       |

## IA & Analytique

| Méthode | Endpoint                 | Rôle                             | Protection      |
|---------|--------------------------|----------------------------------|-----------------|
| POST    | /api/ai/midyear-review   | Génère résumé mi-annuel IA       | auth            |
| POST    | /api/ai/final-review     | Génère évaluation finale IA      | auth            |
| POST    | /api/ai/manager-review   | Génère review manager IA         | auth            |
| POST    | /api/ai/development-plan | Plan de développement IA         | auth            |
| POST    | /api/ai/goal-quality     | Analyse qualité SMART d'objectif | auth            |
| GET     | /api/performance         | Métriques de performance         | auth            |
| GET     | /api/stats               | Statistiques générales           | auth + HR/ADMIN |
| GET     | /api/reports             | Rapports                         | auth + HR/ADMIN |
| GET     | /api/audit-logs          | Journaux d'audit                 | auth + HR/ADMIN |
| GET     | /api/career              | Parcours carrière                | auth            |
| POST    | /api/career              | Créer un parcours                | auth            |
| GET     | /api/pdf                 | Générer un PDF                   | auth            |

## 2.6 Middlewares Globaux et Locaux

### *Middlewares Globaux (dans app.js, appliqués à toutes les routes)*

| Middleware | Librairie | Rôle                                                               |
|------------|-----------|--------------------------------------------------------------------|
| helmet()   | helmet ^8 | Définit les en-têtes HTTP de sécurité (CSP, X-Frame-Options, etc.) |
| cors()     | cors ^2.8 | Autorise uniquement localhost : 5173, 3000, 8081                   |

| Middleware                              | Librairie                           | Rôle                                               |
|-----------------------------------------|-------------------------------------|----------------------------------------------------|
| <code>compression()</code>              | <code>compression ^1.8</code>       | Comprime les réponses en Gzip                      |
| <code>express.json()</code>             | Express natif                       | Parse le corps JSON des requêtes                   |
| <code>xss()</code>                      | <code>xss-clean ^0.1</code>         | Nettoie les entrées pour prévenir les attaques XSS |
| <code>mongoSanitize()</code>            | <code>express-mongo-sanitize</code> | Retire les opérateurs MongoDB (\$, .) des inputs   |
| <code>rateLimit()</code>                | <code>express-rate-limit</code>     | 5000 req / 15 min (skip en local)                  |
| Cache-Control                           | Express natif                       | Désactive le cache HTTP pour les réponses API      |
| <code>express.static('/uploads')</code> | Express natif                       | Sert les fichiers uploadés avec cache 7 jours      |

### ***Middlewares Locaux (appliqués route par route)***

| Middleware                       | Fichier                              | Rôle                                                       |
|----------------------------------|--------------------------------------|------------------------------------------------------------|
| <code>auth</code>                | <code>middleware/auth.js</code>      | Vérifie le JWT Bearer token, peuple <code>req.user</code>  |
| <code>role(...)</code>           | <code>middleware/role.js</code>      | RBAC — vérifie que <code>req.user.role</code> est autorisé |
| <code>validate(schema)</code>    | <code>middleware/validate.js</code>  | Valide <code>req.body</code> avec un schéma Joi            |
| <code>audit(action, type)</code> | <code>middleware/audit.js</code>     | Journalise l'action en base de données                     |
| <code>ownership</code>           | <code>middleware/ownership.js</code> | Vérifie que l'utilisateur est propriétaire de la ressource |

## **2.7 Gestion des Erreurs**

La gestion des erreurs est centralisée dans `middleware/errorHandler.js` :

Erreur levée dans un controller

```

↓
app.use(errorHandler) — middleware de 4 paramètres (err, req, res, next)
↓
if (CastError → ObjectId invalide)    → HTTP 400 "Invalid ID format."
if (ValidationError Mongoose)         → HTTP 400 messages des champs invalides
if (err.statusCode défini)             → HTTP [statusCode] + err.message
else                                   → HTTP 500 "Internal Server Error"
↓
En développement : stack trace incluse dans la réponse JSON
En production    : stack trace omise

```

**Format de réponse d'erreur uniforme :**

```
{
  "success": false,
  "message": "Description de l'erreur",
  "stack": "...uniquement en development..."
}
```

**Non-handled rejections** : `server.js` intercepte `unhandledRejection` et appelle `process.exit(1)`.

**404** : Un handler catch-all retourne `{ success: false, message: "Route not found: /chemin" }`.

## 2.8 Variables d'Environnement Backend

| Variable              | Exemple de valeur            | Usage                                                      |
|-----------------------|------------------------------|------------------------------------------------------------|
| NODE_ENV              | development                  | Mode d'exécution (active/désactive certains comportements) |
| PORT                  | 5009                         | Port d'écoute du serveur Express                           |
| MONGO_URI             | mongodb+srv://...            | URI de connexion à MongoDB Atlas                           |
| JWT_SECRET            | your_super_secret...         | Clé de signature des access tokens JWT                     |
| JWT_REFRESH_SECRET    | your_super_secret_refresh... | Clé de signature des refresh tokens                        |
| JWT_EXPIRE            | 7d                           | Durée de validité des tokens (legacy)                      |
| CORS_ORIGIN           | http://localhost:5173        | Origine autorisée par CORS                                 |
| AI_PROVIDER           | gemini                       | Fournisseur IA actif (gemini/openai/grok)                  |
| GEMINI_API_KEY        | AQ.Ab8RN6...                 | Clé API Google Gemini                                      |
| AI_MODEL              | gemini-2.5-flash             | Modèle IA utilisé                                          |
| AI_TIMEOUT_MS         | 15000                        | Timeout des appels IA (ms)                                 |
| AI_MAX_INPUT_CHARS    | 12000                        | Taille max du contexte envoyé à l'IA                       |
| MIN_OBJECTIVES        | 4                            | Nombre minimum d'objectifs par cycle                       |
| MAX_OBJECTIVES        | 7                            | Nombre maximum d'objectifs par cycle                       |
| TOTAL_OBJECTIVE_VALUE | 70                           | Poids total cible des objectifs                            |
| SMTP_HOST             | smtp.gmail.com               | Serveur SMTP pour les emails                               |
| SMTP_PORT             | 587                          | Port SMTP                                                  |
| SMTP_USER             | moatazben1@gmail.com         | Compte email expéditeur                                    |

| Variable  | Exemple de valeur   | Usage                         |
|-----------|---------------------|-------------------------------|
| SMTP_PASS | uroo xvta isfg prmy | Mot de passe applicatif Gmail |

[!WARNING] Le fichier `.env` est commis dans le dépôt avec des vraies clés. C'est une vulnérabilité de sécurité critique. En production, les secrets doivent être injectés via des variables d'environnement système ou un gestionnaire de secrets (Kubernetes Secrets, HashiCorp Vault).

## 3. ARCHITECTURE FRONTEND

### 3.1 Framework et Outil de Build

- **Framework UI** : React ^18.3.1 avec les Hooks fonctionnels
- **Outil de build** : Vite ^7.2.4 (remplacement de Webpack/CRA — plus rapide grâce à ESM modules natifs)
- **Type** : "type": "module" (ES Modules natifs)
- **Tests** : Jest + Testing Library

### 3.2 Organisation des Composants

```

src/
├─ main.jsx                                ← ReactDOM.createRoot → <App /> dans
<BrowserRouter>
├─ App.jsx                                ← Providers + Routes
├─ routes/
│   └─ routeConfig.jsx                    ← Déclaration centralisée de toutes les routes
├─ components/                            ← Composants réutilisables
│   └─ AuthContext.jsx                    ← Context auth (user, token, login, logout)
│   └─ ThemeContext.jsx                    ← Context thème (darkMode)
│   └─ ActiveCycleContext.jsx              ← Context cycle actif
│   └─ EnterpriseSidebar.jsx               ← Navigation latérale principale
│   └─ TopHeader.jsx                       ← Barre supérieure
│   └─ DashboardLayout.jsx                 ← Layout : Sidebar + Header + <Outlet>
│   └─ ProtectedRoute.jsx                  ← Redirige vers /login si non authentifié
│   └─ RouteGuard.jsx                      ← Contrôle rôle + phase du cycle
│   └─ Notifications.jsx                   ← Centre de notifications
│   └─ UserAvatar.jsx                      ← Avatar utilisateur
│   └─ ProgressChart.jsx                   ← Graphique de progression
│   └─ ai/                                 ← Composants assistants IA
│   └─ common/                             ← Composants partagés (ErrorBoundary,
LoadingSkeleton...)
├─ dashboard/                             ← Widgets du tableau de bord
├─ evaluations/                           ← Composants d'évaluation
├─ goals/                                 ← Composants objectifs
├─ tasks/                                 ← Composants tâches
├─ teams/                                 ← Composants équipes
└─ pages/                                 ← 30 pages (une par domaine fonctionnel)
    └─ Login.jsx
    └─ Dashboard.jsx
    └─ GoalsPage.jsx
    └─ TasksPage.jsx
    └─ MeetingsPage.jsx

```



- MidYearPage.jsx
- FinalEvaluationPage.jsx
- FinalEvaluationEmployee.jsx
- FinalEvaluationManager.jsx
- ManagerReviewPage.jsx
- EvaluationScoringPage.jsx
- EvaluationListPage.jsx
- HRValidation.jsx
- HRDecisions.jsx
- AnalyticsPage.jsx
- BonusPenaltyPage.jsx
- CareerPage.jsx
- CalendarPage.jsx
- FeedbackPage.jsx
- MyTeamPage.jsx
- Teams.jsx
- Users.jsx
- Cycles.jsx
- PerformancePage.jsx
- AuditLogsPage.jsx
- TeamFeed.jsx
- Validation.jsx
- Evaluations.jsx
- Settings.jsx

### 3.3 Gestion du Routing

**Librairie :** React Router DOM ^6.30.3 avec <BrowserRouter> (dans main.jsx).

**Configuration centralisée dans routes/routeConfig.jsx :**

```
// Chargement paresseux (lazy loading) de chaque page
const Dashboard = lazyWithPreload(() => import('../pages/Dashboard'));
// ... etc.
```

**Toutes les routes déclarées :**

| Chemin               | Page                    | Section      | Rôles requis | Phase requise |
|----------------------|-------------------------|--------------|--------------|---------------|
| /login               | Login.jsx               | Auth         | Public       | —             |
| /dashboard           | Dashboard.jsx           | Main         | Tous         | —             |
| /goals               | GoalsPage.jsx           | Main         | Tous         | —             |
| /tasks               | TasksPage.jsx           | Main         | Tous         | —             |
| /calendar            | CalendarPage.jsx        | Main         | Tous         | —             |
| /meetings            | MeetingsPage.jsx        | Main         | Tous         | —             |
| /feed                | TeamFeed.jsx            | Main         | Tous         | —             |
| /cycles              | Cycles.jsx              | Annual Cycle | Tous         | —             |
| /midyear-assessments | MidYearPage.jsx         | Annual Cycle | Tous         | <b>phase2</b> |
| /manager-review      | ManagerReviewPage.jsx   | Annual Cycle | Tous         | —             |
| /final-evaluations   | FinalEvaluationPage.jsx | Annual Cycle | Tous         | <b>phase3</b> |

| Chemin              | Page                      | Section      | Rôles requis     | Phase requise |
|---------------------|---------------------------|--------------|------------------|---------------|
| /evaluation-scoring | EvaluationScoringPage.jsx | Annual Cycle | Tous             | —             |
| /performance        | PerformancePage.jsx       | Annual Cycle | Tous             | —             |
| /my-team            | MyTeamPage.jsx            | People       | Tous             | —             |
| /teams/:id          | MyTeamPage.jsx            | People       | Tous             | —             |
| /feedback           | FeedbackPage.jsx          | People       | Tous             | —             |
| /career             | CareerPage.jsx            | Development  | Tous             | —             |
| /evaluations        | Evaluations.jsx           | Development  | Tous             | —             |
| /validation         | Validation.jsx            | Management   | <b>ADMIN, HR</b> | —             |
| /hr-validation      | HRValidation.jsx          | Management   | <b>ADMIN, HR</b> | —             |
| /hr-decisions       | HRDecisions.jsx           | Management   | <b>ADMIN, HR</b> | —             |
| /teams              | Teams.jsx                 | Management   | <b>ADMIN, HR</b> | —             |
| /users              | Users.jsx                 | Management   | <b>ADMIN, HR</b> | —             |
| /analytics          | AnalyticsPage.jsx         | Management   | <b>ADMIN, HR</b> | —             |
| /audit-logs         | AuditLogsPage.jsx         | Management   | <b>ADMIN, HR</b> | —             |
| /bonus-penalty      | BonusPenaltyPage.jsx      | Management   | <b>ADMIN, HR</b> | —             |
| /settings           | Settings.jsx              | Management   | <b>ADMIN, HR</b> | —             |

### 3.4 Gestion du State Global

**Pas de Redux ni de Zustand.** L'état global est géré via 3 **Context API** React :

#### ***AuthContext.jsx — État d'authentification***

Expose: { user, token, loading, login, logout, updateUser }

- user: objet { \_id, id, name, email, role, profileImage }
- token : access token JWT (string)
- loading : booléen pendant le chargement initial
- login(email, password) : appel POST /api/auth/login, stocke les tokens, met à jour le state
- logout() : appel POST /api/auth/logout, vide localStorage, efface le state

**Stratégie de persistance :** Les tokens et le profil utilisateur sont stockés dans `localStorage` sous les clés `token`, `refreshToken`, `authUser`. Au rechargement de la page, l'état est

rehydraté depuis `localStorage` avant que l'API soit consultée (optimisation UX).

### ***ThemeContext.jsx — Thème sombre/clair***

Expose : `{ darkMode, toggleDarkMode }` — persisté dans `localStorage`.

### ***ActiveCycleContext.jsx — Cycle de performance actif***

Expose : `{ activeCycle, currentPhase, loading }` — interroge l'API pour obtenir le cycle actif, utilisé par `RouteGuard` pour bloquer l'accès aux routes de phase.

## **3.5 Communication avec le Backend**

Deux mécanismes coexistent :

1. **`apiClient.js`** — Instance Axios centralisée dans `src/api/apiClient.js`
2. **`axios global`** — Utilisé directement dans `AuthContext.jsx`

L'instance `apiClient` est configurée avec :

- `baseUrl = import.meta.env.VITE_API_BASE_URL || '/api'`
- Header `Content-Type: application/json`
- **Intercepteur de requête** : attache automatiquement le Bearer token depuis `localStorage`
- **Intercepteur de réponse** : gère les erreurs 401 en tentant un refresh token automatique

**Gestion des erreurs réseau dans `apiClient.js` :**

```
Réponse HTTP 401 reçue (première occurrence)
↓ _retry = true
↓ Récupère le refreshToken depuis localStorage
↓ POST /api/auth/refresh
↓ Si succès → met à jour les tokens → réessaie la requête originale
↓ Si échec → supprime les tokens → redirige vers /login
```

**Gestion dans les composants** : Les erreurs sont attrapées dans des blocs `try/catch` et affichées via des états locaux (ex: `const [error, setError] = useState('')`).

## **3.6 Variables d'Environnement Frontend**

| Variable                           | Valeur                                            | Usage                                                                  |
|------------------------------------|---------------------------------------------------|------------------------------------------------------------------------|
| <code>VITE_API_URL</code>          | <code>/api</code>                                 | Base URL des appels API                                                |
| <code>VITE_DEV_PROXY_TARGET</code> | <code>http://localhost:5009</code>                | Cible du proxy Vite en développement                                   |
| <code>VITE_API_BASE_URL</code>     | Non défini (utilise <code>/api</code> par défaut) | Utilisé dans <code>apiClient.js</code> et <code>AuthContext.jsx</code> |

**Proxy Vite** : En développement, toute requête vers `/api/**` est automatiquement proxifiée vers `http://localhost:5009` (backend). En production Docker, Nginx fait ce rôle.

---

## 4. AUTHENTIFICATION & AUTORISATION

### 4.1 Mécanisme d'Authentification

**JWT (JSON Web Token) avec double-token :**

- **Access Token** : durée courte (8h en prod, 365d en dev), signé avec `JWT_SECRET`
- **Refresh Token** : durée longue (30j en prod), signé avec `JWT_REFRESH_SECRET`, stocké en base de données dans le champ `user.refreshToken`

### 4.2 Flux d'Inscription (Signup)

[!NOTE] Il n'existe **pas de route d'inscription publique** dans le code. Les utilisateurs sont créés par un ADMIN ou RH via `POST /api/users`. Le mot de passe est hashé automatiquement par le hook `pre('save')` de Mongoose.

**Étapes de création d'un utilisateur (par ADMIN/RH) :**

1. L'ADMIN/RH envoie `POST /api/users` avec `{ name, email, password, role, team }`
2. La validation Joi vérifie les champs requis
3. Mongoose valide que l'email se termine par `@biat.com`
4. Le hook `userSchema.pre('save')` déclenche `bcrypt.genSalt(12)` puis `bcrypt.hash(password, salt)`
5. L'utilisateur est sauvegardé avec le mot de passe hashé

### 4.3 Flux de Connexion (Login)

**Étape 1 :** Le frontend envoie `POST /api/auth/login` avec `{ email, password }`

**Étape 2 :** Le backend (`routes/auth.js`, fonction `router.post('/login')`):

- Vérifie que email et password sont présents
- Recherche l'utilisateur : `User.findOne({ email: email.toLowerCase(), isDeleted: false }).select('+password')`
  - `(.select('+password'))` est nécessaire car le champ est `select: false` dans le schéma)
- Appelle `user.comparePassword(password) → bcrypt.compare(candidate, this.password)`
- Si correspondance : génère `accessToken` et `refreshToken` via `generateTokens(user)`
- Sauvegarde le `refreshToken` dans `user.refreshToken` en base
- Retourne `{ success: true, token, accessToken, refreshToken,`

```
user: { _id, name, email, role, profileImage } }
```

**Étape 3 :** Le frontend (`AuthContext.jsx`, fonction `login()`):

- Stocke l'`accessToken` dans `localStorage.setItem('token', ...)`
- Stocke le `refreshToken` dans `localStorage.setItem('refreshToken', ...)`
- Met à jour le state React `user` et `token`
- `navigate('/dashboard')` via React Router

## 4.4 Flux de Déconnexion (Logout)

**Étape 1 :** `AuthContext.logout()` récupère le `refreshToken` depuis `localStorage`

**Étape 2 :** Envoie `POST /api/auth/logout` avec `{ refreshToken }`

**Étape 3 :** Le backend (`routes/auth.js`):

- `User.updateOne({ refreshToken }, { $unset: { refreshToken: 1 } })` — supprime le token côté serveur
- Retourne `{ success: true, message: 'Logged out successfully' }`

**Étape 4 :** Le frontend (`clearSession()`):

- `localStorage.removeItem('token')`
- `localStorage.removeItem('refreshToken')`
- `localStorage.removeItem('authUser')`
- Met le state `user` à `null`, `token` à `null`
- La redirection vers `/login` est automatique via `ProtectedRoute`

[!NOTE] L'`access token` n'est pas invalidé côté serveur (pas de liste noire). Si quelqu'un possède un `access token` valide, il peut l'utiliser jusqu'à son expiration (8h max en prod). C'est acceptable pour la plupart des applications, mais constitue un point à connaître pour le jury.

## 4.5 Protection des Routes Backend

Le middleware `auth.js` est appliqué individuellement sur chaque route protégée :

```
const authMiddleware = require('../middleware/auth');
router.get('/objectives', authMiddleware, objectiveController.getAll);
```

**Fonctionnement de `auth.js` :**

1. Lit `req.headers.authorization` — attend `"Bearer <token>"`
2. Extrait et vérifie le token : `jwt.verify(token, process.env.JWT_SECRET)`
3. Extrait `userId` du payload décodé

4. **Cache en mémoire** (TTL 10 secondes) : vérifie si l'utilisateur est déjà en cache pour éviter des requêtes DB répétées sur des requêtes parallèles
5. Si absent du cache : `User.findById(userId).select('_id role isActive isDeleted').lean()`
6. Vérifie `user.isActive` et `!user.isDeleted`
7. `Peuple req.user = { id, role, _id, ...decoded }`
8. `next()` si tout est valide

**Transmission du token :** Via l'en-tête HTTP Authorization: Bearer <token>.

## 4.6 Protection des Routes Frontend

**Deux composants de protection :**

**ProtectedRoute.jsx** (garde basique) :

```
Si user === null → <Navigate to="/login" replace />
Si loading → spinner
Sinon → render children
```

**RouteGuard.jsx** (garde avancé) :

```
Si !user → <Navigate to="/login" />
Si route.allowedRoles défini && user.role ∉ allowedRoles → Page "accès refusé"
Si route.allowedPhases défini && cycle.currentPhase ∉ allowedPhases → Page
"phase non disponible"
Sinon → render children (la page)
```

## 4.7 Gestion des Rôles et Permissions

**Quatre rôles existants :**

| Rôle         | Description            | Accès                                                        |
|--------------|------------------------|--------------------------------------------------------------|
| ADMIN        | Administrateur système | Accès total — bypass de tous les contrôles de rôle           |
| HR           | Ressources Humaines    | Gestion des utilisateurs, équipes, décisions RH, validation  |
| TEAM_LEADER  | Chef d'équipe          | Gestion de son équipe, approbation des objectifs des membres |
| COLLABORATOR | Collaborateur          | Gestion de ses propres objectifs, check-ins, feedbacks       |

**Middleware role.js :**

```
module.exports = function (...roles) {
  return function (req, res, next) {
    if (req.user.role === 'ADMIN') return next(); // bypass total
    if (!allowedRoles.includes(req.user.role))
```

```
    return res.status(403).json({ success: false, message: 'Forbidden' });
  }
  next();
};
};
```

#### Ressources restreintes :

- Création/gestion utilisateurs & équipes : ADMIN, HR
  - Approbation objectifs : TEAM\_LEADER, HR, ADMIN
  - Validation évaluations finales (HR) : HR, ADMIN
  - Décisions RH (bonus, promotions) : HR, ADMIN
  - Statistiques & analytics : HR, ADMIN
  - Journaux d'audit : HR, ADMIN
- 

## 5. COMMUNICATION FRONTEND ↔ BACKEND

### 5.1 Comment le Frontend Connaît l'URL du Backend

#### En développement :

- VITE\_API\_URL=/api dans frontend/.env
- Le serveur Vite proxifie /api/\*\* vers http://localhost:5009 (configuré dans vite.config.js)
- Le frontend appelle simplement /api/auth/login sans connaître le port du backend

#### En production (Docker) :

- Nginx reçoit les requêtes sur le port 80
- location /api { proxy\_pass http://backend:5000; } dans nginx.conf
- backend est le nom du service dans le réseau Docker interne

#### Dans le code :

```
// apiClient.js
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || '/api';
// AuthContext.jsx
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || '/api';
```

### 5.2 Format des Échanges de Données

- JSON pour toutes les API (header Content-Type: application/json)
- **FormData** / **multipart** pour les uploads de fichiers (check-ins, images de profil) via multer
- Les réponses de succès suivent le format : { success: true, data: {...},

```
message: "..."} }
```

- Les réponses d'erreur suivent le format : { success: false, message: "..."} }

## 5.3 Gestion des Tokens dans les Requêtes

Intercepteur de requête dans `apiClient.js` :

```
apiClient.interceptors.request.use((config) => {  
  const token = localStorage.getItem('token');  
  if (token) {  
    config.headers.Authorization = `Bearer ${token}`;  
  }  
  return config;  
});
```

Intercepteur global dans `AuthContext.jsx` :

```
axios.defaults.headers.common.Authorization = 'Bearer ' + token;
```

Les deux mécanismes coexistent : `apiClient` pour les appels directs, `AuthContext` pour les appels `axios` natifs.

## 5.4 Gestion des Réponses d'Erreur Côté Frontend

Dans `apiClient.js` :

```
401 → tentative de refresh token → si échec → localStorage.clear() → redirect /login  
Autre erreur → throw new Error(response.data.message || message || 'API request failed')  
avec .status et .data attachés
```

**Dans les composants** : Gestion individuelle via `try/catch` et affichage dans des états locaux (`setError(message)`).

## 5.5 WebSockets ou Communication Temps Réel

**Non applicable.** L'application ne comporte pas de communication en temps réel (WebSockets, SSE, etc.). Les notifications sont récupérées par polling à chaque chargement de la page ou navigation. Il n'y a aucune utilisation de `socket.io` ou de toute autre librairie temps réel dans le code source.

---

# 6. FONCTIONNALITÉS MÉTIER PRINCIPALES

## 6.1 Gestion du Cycle Annuel de Performance

**Description** : L'application est structurée autour d'un cycle annuel divisé en 3 phases :

- **Phase 1** : Fixation des objectifs (collaborateurs soumettent, managers approuvent)
- **Phase 2** : Bilan mi-annuel (évaluation intermédiaire, check-ins)
- **Phase 3** : Évaluation finale de fin d'année (évaluation finale, décisions RH)



#### Frontend :

- `PageCycles.jsx` : visualisation et gestion des cycles
- `ActiveCycleContext.jsx` : expose la phase courante à toute l'application
- `RouteGuard.jsx` : bloque l'accès à `/midyear-assessments` hors phase 2, `/final-evaluations` hors phase 3

#### Backend :

- Route : `PUT /api/cycles/:id + PATCH /api/cycles/:id/phase`
  - Contrôleur : `cycleController.js`
  - Modèle : `Cycle.js` avec validation séquentielle des dates de phases dans le hook `pre('save')`
- 

## 6.2 Gestion des Objectifs (SMART Goals)

**Description :** Chaque collaborateur crée 4 à 7 objectifs par cycle avec un poids total de 70 points. Les objectifs suivent un workflow d'approbation :

`draft` → `pending` → `pending_approval` → `approved` → `evaluated` → `archived`

#### Frontend :

- `GoalsPage.jsx` : création, modification, soumission des objectifs
- Composants `goals/` : formulaires, listes, timeline de statut

#### Backend :

- Routes : `POST/GET/PUT/DELETE /api/objectives` + endpoints d'action (`submit`, `approve`, `reject`, `evaluate`)
  - Contrôleur : `objectiveController.js` (85 Ko — le plus volumineux, logique métier dense)
  - Modèle : `Objective.js` avec sous-schémas `KpiSchema`, `ProgressUpdateSchema`, `ChangeRequestSchema`, `ActivityLogSchema`
  - **Calcul automatique :** Hook `pre('save')` calcule `weightedScore = (weight × achievementPercent) / 100`
- 

## 6.3 Assistant IA pour les Reviews

**Description :** Un service IA génère automatiquement des résumés narratifs pour les reviews de performance, les évaluations finales et les plans de développement.

#### Frontend :

- Boutons "Generate with AI" dans `MidYearPage.jsx`, `ManagerReviewPage.jsx`,

`FinalEvaluationManager.jsx`

- Composants dans `components/ai/`

**Backend :**

- Routes : `POST /api/ai/midyear-review, /final-review, /manager-review, /development-plan, /goal-quality`
  - Contrôleur : `aiController.js` (65 Ko)
  - Service : `aiService.js` — abstraction multi-provider (Gemini/OpenAI/Grok)
    - Gemini : utilise `@google/generative-ai` SDK natif
    - OpenAI/Grok : utilise le SDK OpenAI (Grok est compatible OpenAI)
  - Service : `reviewContextService.js` — collecte le contexte (objectifs, scores, historique) avant l'appel IA
  - **Modes IA** : `midyear_summary, final_self_assessment, manager_review, development_plan`
  - **Fallback** : Si l'IA échoue après 2 tentatives, retourne des champs vides avec un message "please complete manually"
- 

## 6.4 Check-ins et Suivi de Progression

**Description** : Les collaborateurs soumettent des check-ins périodiques sur leurs objectifs avec un pourcentage de progression, des notes et des pièces jointes. Les managers valident ou demandent des révisions.

**Frontend :**

- Intégré dans `GoalsPage.jsx` et `ManagerReviewPage.jsx`

**Backend :**

- Routes : `GET/POST/PUT /api/checkins`
  - Contrôleur : `checkInController.js`
  - Modèle : `CheckIn.js` avec historique des soumissions et gestion des pièces jointes (via `multer`)
  - Upload stocké dans `backend/uploads/checkins/`
- 

## 6.5 Évaluations Finales et Workflow RH

**Description** : En fin de cycle (phase 3), les managers créent des évaluations finales avec score calculé, recommandation (promotion, bonus, PIP), validation par les RH et accusé de réception par le collaborateur.

**Frontend :**

- Pages `FinalEvaluationPage.jsx` (routing), `FinalEvaluationManager.jsx` (côté manager), `FinalEvaluationEmployee.jsx` (côté collaborateur)
- Page `HRValidation.jsx` : validation RH des évaluations

**Backend :**

- Routes : `/api/final-evaluations` + `/api/evaluations`
  - Controllers : `finalEvaluationController.js` (25 Ko), `evaluationController.js` (22 Ko)
  - Workflow : `draft` → `pending_hr` → `validated` → `closed`
  - Service : `scoreCalculationService.js` calcule le score automatique pondéré
- 

## 6.6 Gestion des Équipes et des Membres

**Description :** Les ADMIN/HR créent des équipes, y assignent un chef d'équipe (TEAM\_LEADER) et des collaborateurs. Les chefs d'équipe voient les objectifs de leurs membres.

**Frontend :**

- Pages `Teams.jsx`, `MyTeamPage.jsx`

**Backend :**

- Routes : `/api/teams`, `/api/team-members`
  - Controllers : `teamController.js`, dans `routes/teamMembers.js`
  - Modèle : `Team.js` avec support des sous-équipes (`parentTeam`)
- 

## 6.7 Feedback 360°

**Description :** Les utilisateurs peuvent s'envoyer mutuellement des feedbacks (éloge, suggestion, préoccupation) avec des options d'anonymat et de visibilité.

**Frontend :**

- Page `FeedbackPage.jsx`

**Backend :**

- Route : `/api/feedback`
  - Controller : `feedbackController.js`
  - Modèle : `Feedback.js` avec 7 types (praise, suggestion, concern, 360, peer, manager, self)
-

## 6.8 Décisions RH (Bonus, Promotions)

**Description :** Après l'évaluation finale, les RH enregistrent des décisions formelles (récompense, promotion, bonus, coaching, changement de poste, revue de licenciement) avec un score final.

**Frontend :**

- Pages `HRDecisions.jsx`, `BonusPenaltyPage.jsx`

**Backend :**

- Routes : `/api/hr-decisions`, `/api/bonus-penalty`
  - Contrôleur : `hrDecisionController.js`
  - Modèle : `HRDecision.js` (unique par user+cycle)
- 

## 6.9 Parcours Carrière et Développement

**Description :** Chaque collaborateur peut définir un parcours carrière avec un rôle cible, des compétences à développer et un plan d'actions de développement (formation, mentorat, projets).

**Frontend :**

- Page `CareerPage.jsx`

**Backend :**

- Route : `/api/career`
  - Contrôleur : `careerController.js`
  - Modèles : `CareerPath.js`, `Competency.js`, `CareerRecommendation.js`
- 

## 6.10 Gestion des Tâches

**Description :** Un kanban de tâches (inspiré Trello) avec priorités, dates d'échéance, suivi du temps, listes de vérification et Drag & Drop (@dnd-kit).

**Frontend :**

- Page `TasksPage.jsx` (43 Ko — la plus volumineuse)
- Librairie `@dnd-kit` pour le glisser-déposer

**Backend :**

- Route : `/api/tasks`
  - Contrôleur : `taskController.js` (19 Ko)
  - Modèle : `Task.js` avec `checklist[]`, `timeTracking`, `attachments[]`
-

## 6.11 Réunions et Calendrier

**Description :** Planification de réunions 1-on-1 ou d'équipe avec agenda, compte-rendu, points d'action. Intégration d'un calendrier visuel.

**Frontend :**

- Pages `MeetingsPage.jsx` (39 Ko), `CalendarPage.jsx` (26 Ko)

**Backend :**

- Routes : `/api/meetings`, `/api/calendar`
  - Modèle : `Meeting.js`
- 

## 6.12 Notifications

**Description :** Système de notification interne pour tous les événements du workflow (objectif soumis, approuvé, rejeté, évaluation créée, etc.). Plus de 30 types de notifications.

**Frontend :**

- Composant `Notifications.jsx` (18 Ko) dans le `TopHeader`

**Backend :**

- Route : `/api/notifications`
  - Contrôleur : `notificationController.js`
  - Modèle : `Notification.js` avec 30+ types
- 

# 7. SÉCURITÉ

## 7.1 Mesures de Sécurité Présentes dans le Code

| Mesure                     | Implémentation                                                  | Emplacement                            |
|----------------------------|-----------------------------------------------------------------|----------------------------------------|
| Sécurité HTTP headers      | <code>helmet({ crossOriginResourcePolicy: false })</code>       | <code>app.js</code> ligne 29           |
| Protection XSS             | <code>xss-clean</code> — nettoie les inputs HTML                | <code>app.js</code> ligne 45           |
| Protection injection NoSQL | <code>express-mongo-sanitize</code> — retire \$ et . des inputs | <code>app.js</code> ligne 46           |
| Rate Limiting global       | 5000 req/15 min par IP (skip en local)                          | <code>app.js</code> ligne 60-64        |
| Rate Limiting spécifique   | Configurable par route via <code>routeRateLimiter</code>        | <code>middleware/rateLimiter.js</code> |

| Mesure                               | Implémentation                                             | Emplacement                            |
|--------------------------------------|------------------------------------------------------------|----------------------------------------|
| <b>CORS restrictif</b>               | Whitelist :<br>localhost:5173, 3000,<br>8081 uniquement    | app.js ligne 31-41                     |
| <b>Validation des entrées</b>        | Joi — validation de tous les<br>corps de requête           | middleware/<br>validate.js             |
| <b>Authentification JWT</b>          | Vérification signature +<br>expiration + utilisateur actif | middleware/auth.js                     |
| <b>RBAC</b>                          | Vérification du rôle par route                             | middleware/role.js                     |
| <b>Désactivation du cache API</b>    | Cache-Control: no-<br>store sur toutes les routes<br>API   | app.js ligne 48-58                     |
| <b>Validation email</b>              | Regex @biat.com dans le<br>schéma Mongoose + frontend      | models/User.js +<br>Login.jsx          |
| <b>Soft delete</b>                   | isDeleted: true au lieu<br>de supprimer les utilisateurs   | models/User.js                         |
| <b>Journalisation d'audit</b>        | Chaque action sensible est<br>journalisée dans AuditLog    | middleware/audit.js                    |
| <b>Scan de vulnérabilités Docker</b> | Trivy scan des images<br>(CRITICAL+HIGH) dans<br>CI/CD     | .github/workflows/<br>docker-build.yml |
| <b>Scan de secrets</b>               | Gitleaks dans CI/CD avant<br>toute exécution               | .github/workflows/<br>docker-build.yml |

## 7.2 Hashage des Mots de Passe

- **Librairie** : bcryptjs ^2.4.3
- **Nombre de rounds** : 12 (bcrypt.genSalt(12))
- **Implémentation** : Hook pre( 'save' ) de Mongoose — le hashage est automatique et ne peut être contourné
- **Comparaison** : user.comparePassword(candidate) → bcrypt.compare(candidate, this.password)
- **Le mot de passe ne sort jamais de la base** : select: false dans le schéma — il faut explicitement .select( '+password' ) pour le lire

## 7.3 Points Potentiellement Vulnérables

[!WARNING] **Point 1 — Secrets dans le dépôt Git** : Le fichier backend/.env contenant les vrais secrets (MONGO\_URI avec credentials, clés API, mot de passe SMTP) est commis dans le dépôt. Le pipeline CI/CD intègre Gitleaks qui devrait détecter cela. En production, ces secrets doivent être dans des variables d'environnement système ou un Kubernetes Secret.

[!WARNING] **Point 2 — Refresh tokens en clair en base :** Les refresh tokens sont stockés non-hashés dans `user.refreshToken`. Si la base de données est compromise, les refresh tokens sont directement utilisables. La recommandation est de les hasher avant stockage.

[!WARNING] **Point 3 — Access token non-révocable :** Il n'existe pas de liste noire (blocklist) des tokens révoqués. Un token volé reste valide jusqu'à son expiration naturelle (8h en prod).

[!NOTE] **Point 4 — Cache d'authentification en mémoire :** Le middleware `auth.js` maintient un cache in-process (TTL 10s) des profils utilisateurs. En cas de déploiement multi-instance (plusieurs replicas), ce cache n'est pas partagé. La logique de désactivation immédiate d'un utilisateur peut avoir jusqu'à 10s de latence.

[!NOTE] **Point 5 — Rate limit élevé :** 5000 requêtes / 15 min est très permissif. Pour la route `/auth/login`, une limite plus stricte (ex: 5-10 tentatives) serait recommandée pour prévenir le brute-force.

---

## 8. DÉPLOIEMENT & INFRASTRUCTURE

### 8.1 Dockerisation

#### ***Backend — backend/Dockerfile***

```
FROM node:22-alpine          # Image légère Alpine Linux, Node 22
RUN npm install -g npm@latest
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev         # Installation sans devDependencies (production)
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]    # Démarrage direct (pas nodemon)
```

#### ***Frontend — frontend/Dockerfile (Multi-stage build)***

```
# Stage 1 : Build
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build             # Génère le bundle dans /app/dist

# Stage 2 : Serveur de production
FROM nginx:alpine
RUN apk upgrade --no-cache
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

**Avantage du multi-stage :** L'image finale ne contient que Nginx + les fichiers statiques compilés. Node.js et les 300+ Mo de `node_modules` ne sont pas dans l'image de production.

### Nginx (nginx.conf) :

- Sert les fichiers statiques depuis `/usr/share/nginx/html`
- `try_files $uri $uri/ /index.html` — gère le routing côté client (SPA)
- `/api` → proxy vers `http://backend:5000` (résolution DNS interne Docker)
- Cache des assets statiques : 1 an avec `Cache-Control: public, immutable`

## 8.2 Docker Compose

### Fichier `docker-compose.yml` :

```
services:
  backend:
    build: ./backend
    ports:
      - "5001:5000"          # Port externe 5001 → interne 5000
    env_file:
      - ./backend/.env       # Variables d'environnement depuis le fichier .env
  frontend:
    build: ./frontend
    ports:
      - "3000:80"           # Port externe 3000 → Nginx 80
```

[!NOTE] Le Docker Compose ne définit pas de réseau explicite. Les deux services peuvent se joindre via le réseau `default` créé automatiquement. Nginx accède au backend via `http://backend:5000` grâce au DNS interne Docker.

[!WARNING] Il n'y a pas de service MongoDB dans le Docker Compose — l'application utilise MongoDB Atlas (cloud). En cas de déploiement purement local, il faudrait ajouter un service `mongo`.

## 8.3 CI/CD — GitHub Actions

Fichier : `.github/workflows/docker-build.yml`

Déclencheurs : Push sur `main`, `develop`, `qa`, `staging` + Pull Requests

Pipeline en 4 jobs séquentiels :

```
secret-scan —> backend-tests —> frontend-build —> docker —> smoke-test
```

### Job 1 : *secret-scan*

- Outil : **Gitleaks** `gitleaks/gitleaks-action@v2`
- Rôle : Scan du dépôt pour détecter des secrets (clés API, mots de passe) commis par erreur
- Bloquant : Oui — les autres jobs ne s'exécutent que si ce job réussit

### Job 2 : *backend-tests*

- Prérequis : `secret-scan` réussi
- Installe Node 18, restaure le cache `node_modules`, exécute `npm ci` puis `npm test`



(Jest)

- **Cache npm** via `actions/cache@v4` pour accélérer les builds

### ***Job 3 : frontend-build***

- Prérequis : `backend-tests` réussi
- Installe Node 18, construit le frontend (`npm run build`), exécute les tests (`npm test`)

### ***Job 4 : docker***

- Prérequis : `frontend-build` réussi
- Construit les images Docker backend et frontend taguées avec le **SHA Git** (tag immuable)
- Pousse les images sur **Docker Hub** (`moatazbench/pfe-backend`, `moatazbench/pfe-frontend`)
- **GitOps sur develop** : Met à jour `k8s/overlays/dev/kustomization.yaml` avec le nouveau tag et commit + push automatiquement
- **Scan de sécurité Trivy** : Analyse les images Docker pour des vulnérabilités CRITICAL/HIGH

### ***Job 5 : smoke-test***

- Uniquement sur la branche `develop` après push
- Attend 30 secondes le déploiement
- Appelle `GET /health` via `curl` (5 retries) pour vérifier que le backend est opérationnel

## **8.4 Kubernetes**

**Structure :**

```
k8s/
├── base/
│   ├── backend-deployment.yaml
│   ├── frontend-deployment.yaml
│   └── kustomization.yaml
└── overlays/
    ├── dev/
    ├── qa/
    ├── staging/
    └── prod/
```

← Manifests de base (Kustomize)  
← Deployment + Service backend  
← Deployment + Service frontend  
← Liste des ressources  
← Overlay développement  
← Overlay QA  
← Overlay staging  
← Overlay production

**backend-deployment.yaml :**

```
kind: Deployment
spec:
  replicas: 1
  containers:
    - name: backend
      image: moatazbench/pfe-backend      # Tag mis à jour par CI/CD
      ports: [5000]
      envFrom:
        - secretRef: backend-secret      # Secrets injectés depuis K8s Secret
```

```
---
kind: Service
spec:
  type: ClusterIP          # Accessible uniquement dans le cluster
  port: 5000 → 5000
```

### frontend-deployment.yaml :

```
kind: Deployment
spec:
  replicas: 2              # 2 replicas pour la haute disponibilité
  containers:
    - name: frontend
      image: moatazbench/pfe-frontend:latest
      ports: [80]
---
kind: Service
spec:
  type: ClusterIP
  port: 80 → 80
```

### Observations :

- Les Secrets Kubernetes (**backend-secret**) doivent être créés manuellement dans le namespace **pfe** avant le déploiement
- Pas d'Ingress défini dans les fichiers de base (probablement dans les overlays)
- Pas de ressources (CPU/mémoire) définies (à compléter pour la production)
- Pas de Health Checks (liveness/readiness probes) définis (point à améliorer)

## 8.5 ArgoCD

### Fichier k8s/argocd-app.yaml :

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: pfe
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/Moatazbench/PFE.git
    targetRevision: main          # Branche suivie
    path: k8s/base                # Chemin des manifests
  destination:
    server: https://kubernetes.default.svc
    namespace: pfe                # Namespace de déploiement
  syncPolicy:
    automated:
      prune: true                 # Supprime les ressources non présentes dans
Git                               à Git
    selfHeal: true               # Revert les changements manuels non conformes
  syncOptions:
    - CreateNamespace=true       # Crée le namespace si inexistant
```

**Principe GitOps :** ArgoCD surveille le dépôt Git. Quand le CI/CD met à jour **kustomization.yaml** avec un nouveau tag d'image, ArgoCD détecte le changement et déploie

automatiquement la nouvelle version dans Kubernetes. C'est le pattern GitOps : **Git est la source de vérité pour l'état du cluster.**

## 8.6 Monitoring

[!NOTE] **Non configuré dans le code.** Il n'existe aucun fichier de configuration pour Grafana, Prometheus, ou Loki dans le dépôt. Le seul monitoring en place est le smoke-test du CI/CD qui vérifie que l'endpoint `/health` répond. C'est un axe d'amélioration prioritaire.

---

## 9. QUESTIONS DE JURY — AVEC RÉPONSES DÉTAILLÉES

---

### Q1 — Comment fonctionne l'authentification dans votre application ? Décrivez le flux complet de login.

#### Réponse :

Mon application utilise l'authentification **JWT (JSON Web Token)** avec un système à double token : un access token de courte durée et un refresh token de longue durée.

#### Flux complet de connexion :

**Étape 1 — Saisie des identifiants (Frontend)** L'utilisateur saisit son email (format obligatoire `@biat.com`) et son mot de passe dans `Login.jsx`. Le frontend valide d'abord l'email côté client avant d'envoyer quoi que ce soit au serveur.

**Étape 2 — Appel API** La fonction `login()` de `AuthContext.jsx` envoie une requête :

```
POST /api/auth/login
Content-Type: application/json
{ "email": "user@biat.com", "password": "motdepasse" }
```

#### Étape 3 — Traitement côté serveur (`routes/auth.js`)

- Recherche l'utilisateur en base : `User.findOne({ email, isDeleted: false }).select('+password')` (le `.select('+password')` est nécessaire car le mot de passe est exclu par défaut)
- Vérifie le mot de passe : `bcrypt.compare(candidat, hashStocké)` avec 12 rounds de salt
- Si valide : génère deux tokens via `jwt.sign()` :
  - **Access Token** : payload `{ id, role }`, signé avec `JWT_SECRET`, expire en 8h (production)
  - **Refresh Token** : payload `{ id }`, signé avec `JWT_REFRESH_SECRET`, expire en 30j
- Sauvegarde le refresh token dans `user.refreshToken` en base (pour invalidation)

**Étape 4 — Réponse et stockage côté client** Le serveur retourne `{ success: true, token,`

```
accessToken, refreshToken, user: { _id, name, email, role,
profileImage } }.
```

Le frontend stocke dans `localStorage` :

- `token` → access token
- `refreshToken` → refresh token
- `authUser` → profil utilisateur sérialisé

**Étape 5 — Toutes les requêtes suivantes** L'intercepteur `Axios` dans `apiClient.js` attache automatiquement : `Authorization: Bearer <accessToken>`.

Le middleware `auth.js` côté serveur vérifie ce token à chaque requête protégée.

**Renouvellement automatique :** Quand l'access token expire (HTTP 401), l'intercepteur tente automatiquement `POST /api/auth/refresh` avec le refresh token. Si valide, le nouveau token est stocké et la requête originale est réessayée. Sinon, l'utilisateur est redirigé vers `/login`.

---

## Q2 — Comment le frontend communique-t-il avec le backend ? Quel mécanisme utilisez-vous pour sécuriser les requêtes ?

Réponse :

**Communication :** Le frontend utilise **Axios** comme client HTTP. J'ai créé une instance centralisée dans `src/api/apiClient.js` avec `baseURL = '/api'`. En développement, le serveur Vite proxifie les requêtes `/api/**` vers `http://localhost:5009` (backend). En production, Nginx se charge de ce proxy.

Toutes les données sont échangées en **JSON** (`Content-Type: application/json`). Seuls les uploads de fichiers utilisent `multipart/form-data`.

**Sécurisation des requêtes :**

1. **JWT Bearer Token :** Un intercepteur de requête dans `apiClient.js` lit le token depuis `localStorage` et l'ajoute automatiquement à chaque requête :

```
Authorization: Bearer <accessToken>
```

2. **Refresh automatique :** Un intercepteur de réponse gère les erreurs 401 : il tente de renouveler l'access token via `/api/auth/refresh` sans intervention de l'utilisateur.
3. **Côté serveur :** Le middleware `auth.js` vérifie la signature JWT avec `jwt.verify(token, JWT_SECRET)`, vérifie que l'utilisateur est actif et non supprimé.
4. **HTTPS :** En production, HTTPS doit être configuré au niveau de l'Ingress Kubernetes (non présent dans les fichiers de base, mais attendu dans les overlays).
5. **CORS :** Le backend n'accepte que les requêtes provenant des origines autorisées (liste blanche dans `app.js`).

---

### Q3 — Quelle est la structure de votre base de données ? Quels sont vos principaux modèles et leurs relations ?

Réponse :

J'utilise **MongoDB** avec l'ODM **Mongoose**. Ma base de données s'appelle `hr_evaluation` et contient **20 collections**.

**Les 5 modèles centraux et leurs relations :**

```
User ←—— Team (leader, members[])
|
|— Objective (owner, cycle, team)
|   |— sous-documents : kpis[], progressUpdates[], comments[],
changeRequests[]
|   |— CheckIn (objective_id, employee_id)
|
|— Cycle (year unique, status, phase1/2/3 dates)
|
|— Evaluation (employeeId, evaluatorId, cycleId)
|   |— FinalEvaluation (employee_id, cycle_id) [unique par pair]
|
|— HRDecision (user, cycle) [unique par pair]
```

**Relations importantes :**

- Un **Cycle** contient plusieurs **Objectives** (relation 1:N via `objective.cycle`)
- Un **User** appartient à une **Team** et a un **manager** (User) — auto-référence
- Une **FinalEvaluation** est unique par (`employee_id`, `cycle_id`) — contrainte d'unicité composite
- Les sous-documents Mongoose (KPI, ChangeRequest, ActivityLog dans Objective) sont **embarqués** (embedded) plutôt que référencés, car ils n'ont de sens que dans le contexte de l'objectif parent

**Indexation :** J'ai défini des index composites stratégiques pour les requêtes fréquentes :

- { `owner: 1`, `cycle: 1` } sur Objective pour les dashboards
- { `recipient: 1`, `createdAt: -1` } sur Notification pour la liste de notifications paginée
- { `employee_id: 1`, `cycle_id: 1` } UNIQUE sur FinalEvaluation pour garantir l'unicité

---

### Q4 — Comment avez-vous sécurisé les routes de votre API ?

Réponse :

La sécurité des routes repose sur **deux couches de middlewares** combinées :

**Couche 1 — Authentification (`middleware/auth.js`) :** Toutes les routes privées passent par

ce middleware qui :

1. Vérifie la présence de l'en-tête `Authorization: Bearer <token>`
2. Vérifie la signature JWT avec `jwt.verify(token, JWT_SECRET)`
3. Vérifie en base que l'utilisateur est actif et non supprimé
4. Peuple `req.user` avec l'ID et le rôle

**Couche 2 — Autorisation RBAC (middleware/role.js) :** Certaines routes nécessitent un rôle spécifique. Exemple :

```
router.get('/stats', auth, role('ADMIN', 'HR'), statsController.get);
```

L'ADMIN bypassse tous les contrôles de rôle (`if (role === 'ADMIN') return next();`).

**Couche globale de sécurité (dans app.js) :**

- `helmet()` : 15+ en-têtes HTTP de sécurité automatiques
- `xss-clean` : nettoie les données en entrée contre les injections XSS
- `express-mongo-sanitize` : retire les opérateurs MongoDB (`$where`, `$gt`, etc.) pour prévenir les injections NoSQL
- `express-rate-limit` : 5000 req/15 min par IP

**Validation des données :** Le middleware `validate(schema)` utilise Joi pour valider le corps de chaque requête avant qu'il atteigne le controller.

**Audit :** Le middleware `audit(action, resourceType)` journalise toutes les actions sensibles dans la collection `audit logs`.

---

## Q5 — Expliquez votre architecture globale : pourquoi ce découpage frontend/backend ?

**Réponse :**

J'ai choisi une architecture **découplée (Decoupled Architecture)** avec un frontend et un backend séparés, communiquant via une API REST. Ce choix s'appelle aussi une **SPA + API architecture**.

**Pourquoi ce découpage ?**

1. **Séparation des responsabilités :** Le backend se concentre exclusivement sur la logique métier, l'accès aux données et la sécurité. Le frontend gère uniquement l'interface utilisateur et l'expérience utilisateur. Chacun peut évoluer indépendamment.
2. **Scalabilité indépendante :** En production Kubernetes, le frontend a 2 replicas (charge statique) et le backend 1 replica. Si l'API est surchargée, je peux augmenter uniquement les replicas du backend sans toucher au frontend.
3. **Réutilisabilité de l'API :** La même API backend pourrait servir une application mobile (React Native), un outil CLI, ou une intégration tierce sans modification.

4. **Déploiement optimisé** : Le frontend est compilé en fichiers statiques (HTML/CSS/JS) servis par Nginx, qui est extrêmement performant pour ce cas d'usage. Le backend Node.js gère uniquement les requêtes dynamiques.
5. **Équipe parallèle** : Une équipe frontend et une équipe backend peuvent travailler en parallèle sur le même produit en se mettant d'accord uniquement sur le contrat d'API.

#### Flux d'une requête en production :

```
Navigateur → Nginx (frontend:80)
└─ /index.html, *.js, *.css → fichiers statiques (cache 1 an)
   /api/** → proxy_pass → backend:5000 → MongoDB Atlas
```

---

## Q6 — Comment gérez-vous les erreurs dans votre application (côté serveur et côté client) ?

### Réponse :

#### Côté serveur — Architecture centralisée :

J'utilise un **middleware global de gestion d'erreurs** (`middleware/errorHandler.js`) qui est le **dernier middleware** enregistré dans `app.js`. Tous les controllers peuvent passer leurs erreurs à ce middleware via `next(err)`.

Le handler distingue plusieurs types d'erreurs :

- `CastError` (ObjectId invalide) → HTTP 400 "Invalid ID format."
- `ValidationError` Mongoose → HTTP 400 avec la liste des champs invalides
- Erreurs avec `statusCode` personnalisé → HTTP [statusCode] + message
- Autres erreurs → HTTP 500 "Internal Server Error"

En mode développement, la `stack trace` est incluse dans la réponse pour faciliter le débogage. En production, elle est omise pour ne pas exposer l'architecture interne.

```
// Format uniforme de toutes les erreurs
{ "success": false, "message": "Description de l'erreur" }
```

Les **rejections non-gérées** (`unhandledRejection`) sont interceptées au niveau de `server.js` et provoquent l'arrêt du processus avec `process.exit(1)`.

#### Côté client — Gestion multi-niveaux :

1. **Intercepteur Axios** (`apiClient.js`) : Centralise la gestion des erreurs réseau. Les erreurs 401 déclenchent le refresh automatique. Les autres erreurs sont reformatées en objets `Error` avec les propriétés `.status` et `.data`.
2. **Gestion locale dans les composants** : Chaque page maintient un état `error` local :

```
const [error, setError] = useState('');
try { await apiCall(); } catch(err) { setError(err.message); }
```

Les messages d'erreur sont affichés dans l'interface utilisateur (bannières rouges, toasts).

3. **ErrorBoundary** (`components/common/ErrorBoundary.jsx`) : Composant React qui capture les erreurs d'exécution dans l'arbre de rendu et affiche une UI de fallback au lieu de faire planter toute l'application.
- 

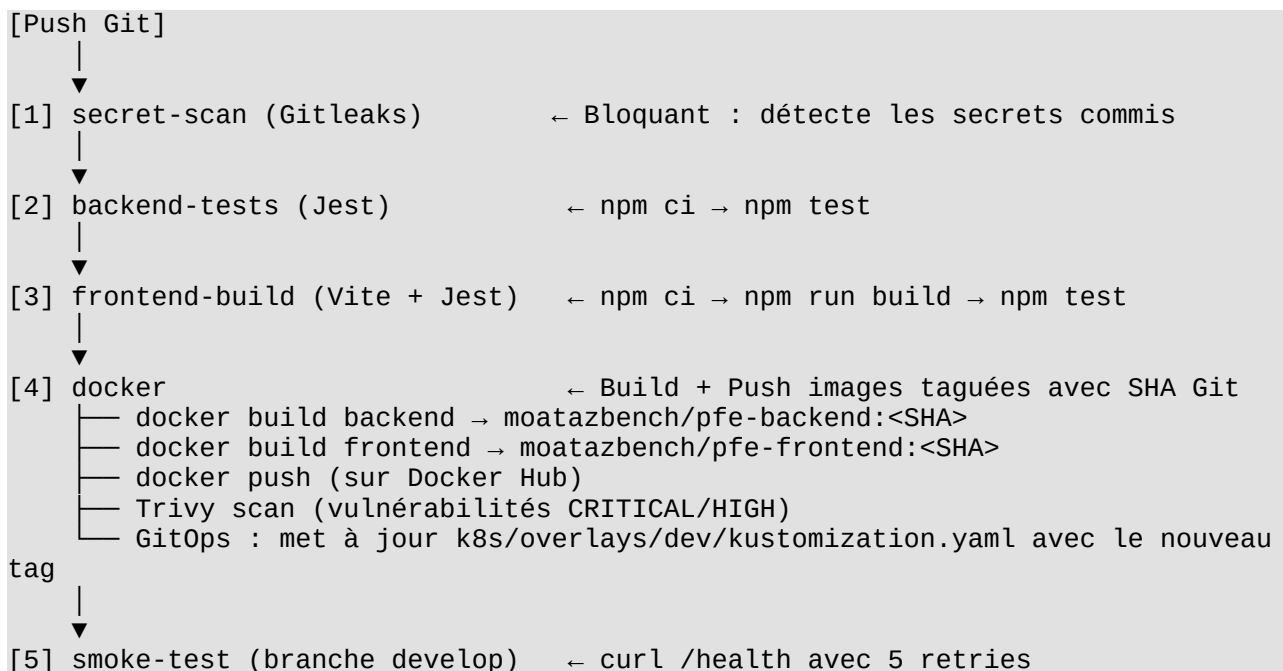
## Q7 — Comment votre application est-elle déployée ? Décrivez votre pipeline CI/CD.

### Réponse :

Mon pipeline de déploiement suit l'approche **GitOps** et utilise : **GitHub Actions** (CI), **Docker Hub** (registry d'images), **Kubernetes** (orchestration), et **ArgoCD** (déploiement continu).

### Pipeline GitHub Actions (`.github/workflows/docker-build.yml`) :

Déclenché sur chaque push sur les branches `main`, `develop`, `qa`, `staging` :



**Déploiement Kubernetes (ArgoCD)** : ArgoCD surveille le dépôt Git sur la branche `main`. Quand le CI/CD met à jour `kustomization.yaml` avec le nouveau SHA d'image, ArgoCD détecte le diff et applique automatiquement les nouveaux manifests au cluster Kubernetes (`kubectl apply`).

ArgoCD est configuré avec :

- `prune: true` : supprime les ressources orphelines
- `selfHeal: true` : révert les changements manuels non conformes à Git

**Environnements** : 4 overlays Kustomize (`dev`, `qa`, `staging`, `prod`) permettent d'avoir des configurations différentes par environnement en surchargeant la base.

---



## Q8 — Pourquoi avez-vous choisi MongoDB pour ce projet ?

### Réponse :

Le choix de **MongoDB** était motivé par les caractéristiques spécifiques de ce projet :

**1. Flexibilité du schéma pour des structures complexes et évolutives :** Le modèle *Objective* est un excellent exemple : il contient des sous-documents imbriqués (`kpis[]`, `progressUpdates[]`, `comments[]`, `changeRequests[]`, `activityLog[]`) qui varient selon le type d'objectif et le stade du workflow. En SQL, cela aurait nécessité au minimum 6 tables avec des jointures complexes. En MongoDB, c'est une seule collection avec des documents flexibles.

**2. Documents embarqués = performance :** Les données fréquemment consultées ensemble sont stockées ensemble. Afficher un objectif avec ses KPIs, ses commentaires et son journal d'activité ne nécessite **aucune jointure** — c'est un seul accès disque.

**3. Scalabilité horizontale :** MongoDB Atlas offre un sharding natif pour distribuer les données sur plusieurs nœuds si la charge augmente, sans modification du code applicatif.

**4. Mongoose + JavaScript :** Dans un contexte MERN, utiliser Mongoose permet de partager des concepts (objets JavaScript) entre le frontend et le backend. Les schémas Mongoose apportent la validation, les hooks (pre/post) et les méthodes que les bases relationnelles offrent via des triggers et procédures stockées.

**5. MongoDB Atlas :** Hébergement cloud géré — pas de serveur de base de données à maintenir, sauvegardes automatiques, monitoring intégré, certificats TLS automatiques.

### Trade-offs acceptés :

- Pas de transactions ACID multi-documents par défaut (MongoDB 4+ supporte les transactions, mais je ne les utilise pas)
- Pas de contraintes de clé étrangère au niveau base — la cohérence est gérée par l'application (Mongoose)
- Risque de dénormalisation si les données partagées évoluent (ex: le nom d'un utilisateur dupliqué dans des logs)

---

## Q9 — Comment fonctionne la gestion des rôles et des permissions dans votre application ?

### Réponse :

J'ai implémenté un **RBAC (Role-Based Access Control)** — contrôle d'accès basé sur les rôles — à deux niveaux : backend et frontend.

### Les 4 rôles :

| Rôle  | Qui                    | Ce qu'ils peuvent faire                         |
|-------|------------------------|-------------------------------------------------|
| ADMIN | Administrateur système | Tout faire — bypass total de tous les contrôles |

| Rôle         | Qui                 | Ce qu'ils peuvent faire                                        |
|--------------|---------------------|----------------------------------------------------------------|
| HR           | Ressources Humaines | Gérer utilisateurs, équipes, valider évaluations, décisions RH |
| TEAM_LEADER  | Chef d'équipe       | Approuver les objectifs de son équipe, mener des reviews       |
| COLLABORATOR | Employé             | Gérer ses propres objectifs, check-ins, feedbacks              |

### Backend — Middleware `role.js` :

```
// Exemple d'utilisation dans une route
router.get('/users', auth, role('ADMIN', 'HR'), userController.getAll);
```

Le middleware `role.js` vérifie que `req.user.role` est dans la liste des rôles autorisés. L'ADMIN bypass toujours (`if (role === 'ADMIN') return next();`). Sinon, HTTP 403 "Forbidden: Insufficient permissions".

Le rôle est lu depuis le token JWT (payload { `id`, `role` }) et vérifié en base à chaque requête via le middleware `auth.js`.

### Frontend — `RouteGuard.jsx` :

Le composant `RouteGuard` vérifie le rôle de l'utilisateur connecté pour les routes avec `allowedRoles` défini :

```
if (route.allowedRoles && !route.allowedRoles.includes(user.role)) {
  return <GuardState title="Page non disponible pour votre rôle" />;
}
```

Exemple : `/validation`, `/hr-decisions`, `/users`, `/teams`, `/analytics`, `/audit-logs`, `/bonus-penalty`, `/settings` sont réservés à ADMIN et HR.

**Contrôle de phase du cycle** : En plus du rôle, certaines routes sont conditionnées à la phase active du cycle de performance. Par exemple, `/midyear-assessments` n'est accessible qu'en `phase2` du cycle. Ce contrôle est également géré par `RouteGuard` via `ActiveCycleContext`.

## Q10 — Si vous deviez améliorer votre application, quels seraient vos 3 axes prioritaires et pourquoi ?

Réponse :

### Axe 1 — Observabilité et Monitoring (Haute priorité)

Actuellement, l'application n'a aucun monitoring. En production, je ne peux pas savoir si des erreurs surviennent, si les performances se dégradent, ou si des anomalies de sécurité se produisent.

**Amélioration concrète** : Intégrer la stack **Prometheus + Grafana + Loki** :

- `prom-client` côté Node.js pour exposer des métriques (latence des routes, taux d'erreurs, connexions MongoDB)
  - Grafana pour la visualisation avec des alertes automatiques (ex: taux d'erreur > 5%)
  - Loki pour l'agrégation centralisée des logs de tous les pods Kubernetes
- 

## Axe 2 — Sécurité des Secrets et Hardening (Haute priorité)

Le fichier `.env` avec les vraies credentials est commis dans le dépôt Git. C'est une vulnérabilité critique qui doit être corrigée avant tout déploiement en production réelle.

### Amélioration concrète :

- Remplacer le `.env` commis par des **Kubernetes Secrets** ou **HashiCorp Vault**
  - Hasher les refresh tokens avant stockage en base
  - Implémenter une liste noire de tokens révoqués (avec Redis comme cache) pour permettre une révocation immédiate
  - Ajouter un rate limiting strict sur `/api/auth/login` (5 tentatives max / 15 min)
- 

## Axe 3 — Notifications Temps Réel et WebSockets (Priorité Moyenne)

Actuellement, les notifications sont récupérées uniquement lors des navigations de page. Les utilisateurs ne voient pas les nouvelles notifications sans recharger manuellement.

**Amélioration concrète :** Implémenter **Socket.IO** (ou Server-Sent Events) pour les notifications temps réel :

- Le backend émet un événement `new_notification` quand une notification est créée
  - Le frontend (connecté via WebSocket) reçoit la notification instantanément et met à jour le badge de notification sans polling
  - Cela améliorerait significativement l'expérience utilisateur pour les workflows d'approbation d'objectifs
- 

# 10. CE QUE LE DÉVELOPPEUR DOIT MAÎTRISER

## Concept 1 : JWT (JSON Web Token) — Structure et Cycle de Vie

**Ce que c'est :** Un JWT est un token signé composé de 3 parties Base64 : `header.payload.signature`. Il encode des informations (claims) vérifiables sans consulter la base de données.

**Dans ce projet :** Le payload contient `{ id, role, iat, exp }`. Le middleware `auth.js` utilise `jwt.verify(token, secret)` pour vérifier la signature et l'expiration. Le rôle dans le token évite une requête DB supplémentaire pour le contrôle d'accès.

**À savoir pour le jury :** Différence entre access token (court, sans état) et refresh token (long, avec état en DB). Pourquoi on ne peut pas "invalidier" un JWT avant son expiration. Comment la rotation des refresh tokens prévient la réutilisation de tokens volés.

---

## Concept 2 : Mongoose Schemas — Validation, Hooks et Relations

**Ce que c'est :** Mongoose est un ODM qui ajoute une couche de schéma typé au-dessus de MongoDB. Il permet de définir des types, des validateurs, des hooks (pre, post) et des méthodes personnalisées.

**Dans ce projet :** Le hook `pre( 'save' )` dans `User.js` hashé automatiquement le mot de passe. Le hook `pre( 'save' )` dans `Objective.js` calcule le `weightedScore`. Les sous-documents (KPI, ChangeRequest) sont embarqués directement dans le document `Objective`.

**À savoir pour le jury :** Différence entre références (`ObjectId`) et documents embarqués, et quand choisir l'un ou l'autre. Comment les index MongoDB améliorent les performances des requêtes fréquentes.

---

## Concept 3 : React Context API — Gestion d'État Global

**Ce que c'est :** La Context API permet de partager de l'état entre des composants sans passer des props à chaque niveau de l'arbre (prop drilling).

**Dans ce projet :** `AuthContext` expose `{ user, token, login, logout }` à tous les composants. `ThemeContext` expose `{ darkMode, toggleDarkMode }`. `ActiveCycleContext` expose la phase du cycle actif.

**À savoir pour le jury :** Pourquoi Context API plutôt que Redux ? Pour une application de cette taille (3 contexts, état peu complexe), Context + hooks est suffisant. Redux apporte de la valeur pour des états très complexes avec de nombreuses mutations.

---

## Concept 4 : Axios Intercepteurs — Automatisation des Requêtes

**Ce que c'est :** Les intercepteurs Axios permettent d'injecter de la logique avant l'envoi d'une requête ou après la réception d'une réponse, de manière centralisée.

**Dans ce projet :** L'intercepteur de requête attache le JWT à chaque appel. L'intercepteur de réponse gère les 401 en tentant un refresh automatique. Sans intercepteurs, il faudrait répéter cette logique dans chaque appel API.

**À savoir pour le jury :** Pattern de "transparent token refresh" — le composant n'a pas conscience que son access token a expiré et a été renouvelé. Les 3 entités : `config`, `request`, `response`.

---

## Concept 5 : RBAC (Role-Based Access Control)

**Ce que c'est :** Contrôle d'accès basé sur des rôles prédéfinis. Chaque utilisateur a un rôle, chaque

ressource/action est associée à un ou plusieurs rôles autorisés.

**Dans ce projet :** 4 rôles (ADMIN, HR, TEAM\_LEADER, COLLABORATOR). Le middleware `role.js` côté serveur + `RouteGuard.jsx` côté client appliquent les contrôles. L'ADMIN bypass toujours.

**À savoir pour le jury :** Différence RBAC / ABAC (Attribute-Based). Pourquoi le contrôle côté frontend seul est insuffisant (UX), et pourquoi le contrôle côté backend seul est le vrai enforcement de sécurité.

---

## Concept 6 : Cycle Annuel de Performance à 3 Phases

**Ce que c'est :** L'application modélise un processus RH annuel structuré en 3 phases temporelles avec un workflow d'approbation à chaque étape.

**Dans ce projet :** Le modèle `Cycle` stocke les dates de début/fin de chaque phase. `ActiveCycleContext` détermine la phase courante. `RouteGuard` bloque l'accès aux pages de phase inappropriée. Les statuts des objectifs (`draft` → `approved` → `evaluated`) correspondent aux étapes du cycle.

**À savoir pour le jury :** Comment les dates de phases sont validées (hook Mongoose séquentiel). Comment la phase courante conditionne les fonctionnalités disponibles.

---

## Concept 7 : GitOps et ArgoCD

**Ce que c'est :** GitOps est un pattern où Git est la source de vérité pour l'état de l'infrastructure. ArgoCD synchronise en permanence l'état du cluster Kubernetes avec ce qui est déclaré dans Git.

**Dans ce projet :** Quand le CI/CD pousse un nouveau tag d'image dans `kustomization.yaml`, ArgoCD détecte la différence et applique le nouveau déploiement. `selfHeal: true` signifie qu'un `kubectl apply` manuel sera annulé par ArgoCD.

**À savoir pour le jury :** Pull-based vs Push-based CD. Pourquoi GitOps améliore l'auditabilité (chaque déploiement a un commit Git associé). Rôle de Kustomize (overlays par environnement sans duplication).

---

## Concept 8 : Docker Multi-Stage Build

**Ce que c'est :** Une image Docker construite en plusieurs étapes où seule la dernière étape est l'image finale. Les étapes précédentes servent à compiler/builder sans être incluses dans l'image de production.

**Dans ce projet :** Le Dockerfile frontend utilise `node:18-alpine` pour compiler le React (étape `builder`) puis `nginx:alpine` pour servir les fichiers statiques. L'image finale ne contient pas Node.js ni les `node_modules` — elle est beaucoup plus petite et plus sécurisée.

**À savoir pour le jury :** Réduction de la taille d'image (plusieurs centaines de Mo économisés). Réduction de la surface d'attaque (moins de logiciels = moins de vulnérabilités). Différence entre

COPY --from=builder et un build classique.

---

## Concept 9 : Google Gemini AI — Intégration d'un LLM dans une Application

**Ce que c'est :** Un Large Language Model (LLM) est un modèle d'IA capable de générer du texte cohérent à partir d'un prompt. L'application l'utilise pour automatiser la rédaction des reviews de performance.

**Dans ce projet :** `aiService.js` est une abstraction multi-provider (Gemini/OpenAI/Grok) configurable via `AI_PROVIDER`. Il envoie un contexte structuré (objectifs, scores, commentaires) au LLM et reçoit une réponse JSON structurée. Un mécanisme de fallback garantit que l'application reste fonctionnelle même si l'IA est indisponible.

**À savoir pour le jury :** Concept de "prompt engineering" (construction du prompt système + utilisateur). Pourquoi les réponses sont demandées en JSON. Limitation : les LLMs peuvent "halluciner" — c'est pourquoi le service demande explicitement "Use ONLY supplied data. No invented facts."

---

## Concept 10 : Middleware Express — Pattern de Pipeline

**Ce que c'est :** Dans Express.js, une requête HTTP passe séquentiellement à travers une chaîne de middlewares avant d'atteindre le handler final. Chaque middleware peut modifier la requête/réponse ou stopper la chaîne avec `next()`.

**Dans ce projet :** La chaîne pour une route protégée est :

```
[Request] → helmet → cors → xss-clean → mongoSanitize → rateLimit → auth.js → role.js → validate.js → controller → errorHandler
```

Chaque maillon a une responsabilité unique. Si l'authentification échoue, `auth.js` retourne une réponse 401 et n'appelle pas `next()` — le controller n'est jamais atteint.

**À savoir pour le jury :** Différence middleware global (monté sur `app.use()`) vs local (monté sur une route spécifique). Le middleware d'erreur doit avoir 4 paramètres (`err`, `req`, `res`, `next`) pour être reconnu par Express.

---

*Fin du rapport — Document généré à partir de l'analyse complète du code source du dépôt Moatazbench/PFE*